

Compute-Communication Overlap Is Not Free: A Cross-Layer Characterization in GPU LLM Workloads

Jihwan Oh*
Stanford University
Stanford, CA, USA
jihwanoh@stanford.edu

Junkyum Kim
Georgia Institute of Technology
Atlanta, GA, USA
jun-kyum.kim@gatech.edu

Seokjin Go
Georgia Institute of Technology
Atlanta, GA, USA
seokjin.go@gatech.edu

Jongse Park
Korea Advanced Institute of Science and Technology
Daejeon, Republic of Korea
jspark@casys.kaist.ac.kr

Divya Mahajan
Georgia Institute of Technology
Atlanta, GA, USA
divya.mahajan@gatech.edu

Abstract

Distributed large language model (LLM) systems rely on compute-communication (CC) overlap to hide communication overhead. However, scheduling compute and communication concurrently and maximizing overlap opportunities do not guarantee effective overlap. Resource contention can slow concurrent operations, while communication that exceeds the available computation time remains exposed on the critical path. We study these limitations using end-to-end LLaMA 3 and Mixtral workloads on single-node H100 and H200 systems and multi-node H100 systems, complemented by targeted microbenchmarks. On a single node, concurrent execution slows kernels by 20.4–55.8% relative to sequential execution, substantially reducing the expected benefit of overlap. In multi-node workloads, kernel slowdown can be modest, but execution is still up to $2.00\times$ slower than an ideal overlap because communication often outlasts computation.

Our microbenchmarks trace these inefficiencies to interactions across the hardware, runtime, and kernel layers. The main contributors are contention for streaming multiprocessors and memory bandwidth, limitations in launch ordering and stream priorities, amplification of stragglers across ranks, and communication kernels that rely heavily on polling. These results show that future systems must optimize CC overlap efficiency, not only overlap opportunity, through coordinated runtime, kernel, and hardware support.

1. Introduction

Recent years have seen the widespread adoption of deep learning across natural language processing, computer vision, and scientific computing [1–3]. Guided by scaling laws [4] that link model scale with performance, modern large language models (LLMs) have grown rapidly in size and complexity, reaching trillions of parameters [5–9]. This growth makes distributed execution across multiple GPUs and GPU servers essential, driving the development of parallelization strategies including data [10, 11], tensor [5],

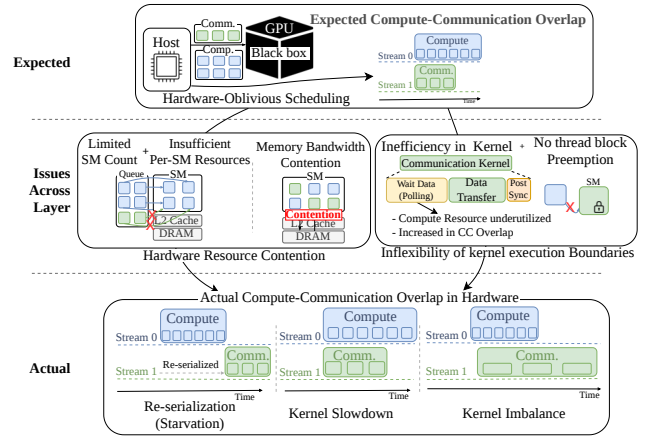


Figure 1: Overview of how expected CC overlap can degrade into re-serialization, kernel slowdown, and kernel imbalance due to hardware-oblivious scheduling, hardware resource contention, and inflexible kernel execution boundaries.

sequence [12], pipeline [13], and expert parallelism [14]. As these strategies require tight coordination across devices, inter-GPU communication can become a performance bottleneck limiting overall system scalability [15–17].

To hide this communication overhead, modern distributed LLM systems, both training and inference, widely adopt *compute-communication (CC) overlap* [7, 18, 19], where communication is arranged to proceed concurrently with computation. While common runtime implementations exploit separate CUDA streams, recent approaches also construct overlap through kernel fusion and other fine-grained mechanisms. A substantial body of prior work proposes optimizations at the kernel level [20, 21], system level [17, 22–26], and hardware level [27] to maximize overlap opportunities.

However, recent work shows that maximizing overlap opportunities alone is insufficient: realized overlap consistently falls short of its theoretical potential and can suffer from resource-contention overhead [27–30]. We argue this

*Work done at SCAI Lab at Georgia Institute of Technology.

is not an incidental engineering gap but a consequence of three compounding issues that are insufficiently addressed:

(1) Hardware-oblivious scheduling. Frameworks such as PyTorch schedule compute and communication kernels according to logical data dependencies, with limited awareness of their GPU resource requirements. As a result, overlap is treated largely as a binary scheduling decision: kernels either execute concurrently or they do not. This approach does not account for whether the kernels can share the GPU efficiently. A schedule may therefore create an overlap opportunity while delivering little or no latency benefit.

(2) Contention for shared GPU resources. Compute and communication kernels do not execute in isolated resource domains. When launched concurrently, they compete for streaming multiprocessors (SMs), memory bandwidth, and on-chip caches [27, 29, 30]. This contention is especially severe because high-performance GEMM and communication kernels are generally optimized to maximize standalone throughput, leaving little resource headroom for concurrent work. Consequently, overlap can slow one or both kernels enough to offset the time saved by executing them concurrently.

(3) Inflexibility of kernel execution boundaries. Even when contention is understood, the GPU execution model limits the ability to address it in software. GPU execution is organized around discrete kernel launches with well-defined boundaries. While kernel fusion can reduce launch overhead and improve locality, monolithic kernels lack the scheduling flexibility needed to dynamically interleave compute and communication at fine granularity. This structural rigidity means that software-level decomposition, however carefully engineered, cannot fully escape the contention imposed by the hardware.

Together, these issues show that CC overlap must be evaluated not only by how many overlap opportunities are created, but also by how efficiently those opportunities are realized. In this paper, we emphasize the importance of CC overlap efficiency and analyze the causes of overlap inefficiency together with insights for improving it. To do so, we conduct end-to-end experiments with LLaMA 3 and Mixtral models across FSDP, TP/SP, inference prefill, and MoE workloads on single-node 8-GPU H100/H200 systems and two-node H100 systems. We then use targeted microbenchmarks for detailed analysis, pairing representative GEMM and non-GEMM compute kernels with all-gather, reduce-scatter, and all-to-all communication.

The end-to-end results quantify how far realized CC overlap can fall short of the expected performance improvement. In the single-node LLM workloads, contention between concurrently executing compute and communication kernels leaves overlapped execution up to $4.46\times$ slower than the idealized overlap time and introduces up to 55.8% kernel-level slowdown relative to sequential execution. Furthermore, the multi-node results reveal a complementary source of inefficiency. Even when kernel-level slowdown is near zero or small, overlapped execution can still remain up to $2.00\times$ slower than ideal because limited inter-node communication bandwidth and communication-progress

imbalance prevent communication from being fully hidden, exposing residual communication time on the critical path. Our detailed analysis decomposes these inefficiencies across the hardware, system, and kernel levels. At the hardware level, SM residency limits can re-serialize overlap and memory contention slows co-resident kernels. At the system level, representative single-node FSDP LLaMA traces show rank-wise straggler amplification, while launch order and stream priority determine whether issued overlap becomes effective concurrent execution. At the kernel level, communication kernels spend more time in polling and peer-wait phases under overlap.

These findings motivate a contention-aware, cross-layer approach to designing future distributed LLM systems. This paper makes the following contributions:

- We characterize CC overlap efficiency in end-to-end LLaMA 3 and Mixtral workloads across single-node and multi-node GPU systems.
- We identify and quantify two distinct sources of lost overlap benefit: contention-induced kernel slowdown within a node and insufficient compute duration to hide communication across nodes.
- We extract representative CC overlap patterns from commonly used LLM parallelization strategies and develop targeted microbenchmarks to isolate their behavior.
- We trace overlap inefficiency to factors across the hardware, runtime, and kernel layers, including SM and memory-bandwidth contention, kernel launch ordering, stream-priority limitations, cross-rank straggler amplification, and polling-heavy communication kernels.
- Based on these findings, we derive cross-layer design principles for improving CC overlap through coordinated runtime, kernel, and hardware support.

2. Background

2.1. Collective Communication in LLMs

Modern LLM workloads commonly rely on sharding-based parallelism to distribute model states, activations, and routed tokens across GPUs. In these schemes, three collective primitives repeatedly appear: all-gather, reduce-scatter, and all-to-all. All-gather and reduce-scatter move sharded tensors in dense-model parallelism, while all-to-all redistributes routed tokens in expert-parallel MoE workloads.

All-gather (AG) collects tensor shards so every rank can consume a replicated view. **Reduce-scatter (RS)** reduces partial results across ranks and returns only the reduced shard to each rank. **All-to-all (A2A)** shuffles per-destination pieces so routed tokens or activations move to the ranks that need them. These collectives form the communication kernels overlapped with GEMM and non-GEMM computation in the later analysis.

Fig. 2 summarizes these data movements, which our experimental workloads issue through NCCL on NVIDIA GPUs [31].

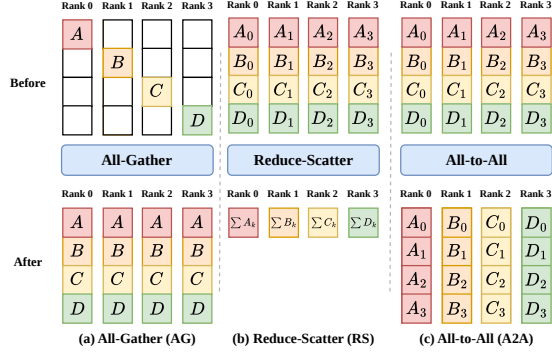


Figure 2: Collective communication primitives.

2.2. Straggler Effect

Collective communication requires global synchronization across participating ranks, so a collective can finish only after the slowest rank arrives and makes progress. This rank-induced delay is commonly referred to as the straggler effect [32–34]. Prior work studies stragglers caused by persistent hardware, workload, or system imbalance. Our analysis focuses on a narrower question: whether CC overlap itself widens the rank-wise collective duration gap when communication executes concurrently with compute kernels.

2.3. Distributed Parallelism Schemes

Modern LLMs combine multiple parallelism strategies to fit model states and activations in GPU memory while maintaining high throughput [5, 7, 11, 12, 14, 18]. We focus on three widely used sharding dimensions: model-state sharding in FSDP, activation and operator sharding in TP/SP, and token/expert sharding in EP.

Fully Sharded Data Parallelism. Fully Sharded Data Parallelism (FSDP) shards model states across data-parallel ranks following ZeRO-style optimizations [11, 18]. Because each rank stores only a shard of a layer’s parameters, FSDP uses parameter all-gather to materialize full weights before computation and gradient reduce-scatter to keep gradients sharded after backward computation.

Tensor and Sequence Parallelism. Tensor parallelism partitions transformer operators across ranks, while sequence parallelism shards activations along the sequence dimension [5, 12]. Together, TP/SP uses activation all-gather when an operator needs a full input view and reduce-scatter to return partial results to a sharded activation layout.

Expert Parallelism. Mixture-of-experts (MoE) models route each token to a subset of experts, increasing model capacity without activating every parameter for every token [14]. Expert parallelism distributes experts across ranks, so routed tokens are exchanged with an all-to-all dispatch before expert feed-forward computation and an all-to-all combine afterward.

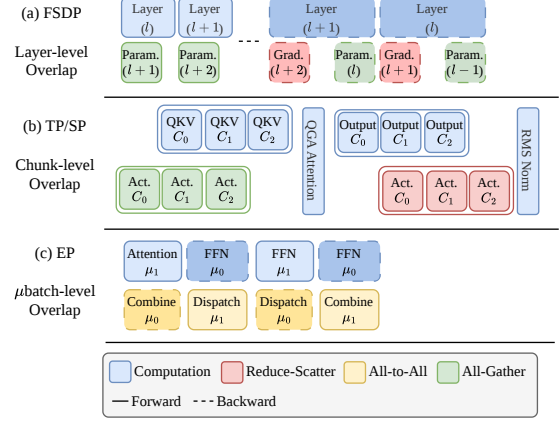


Figure 3: Representative CC overlap patterns in (a) FSDP, (b) TP/SP, and (c) EP.

2.4. CC Overlap Patterns

The collectives introduced by FSDP, TP/SP, and EP are often overlapped with computation, but the dependency structure differs across various schemes. Fig. 3 summarizes the patterns used in this paper. FSDP exposes *independent overlap*: a parameter all-gather or gradient reduce-scatter from one layer can overlap with compute from a neighboring layer because the two operations are on different dependency chains [11, 18]. TP/SP exposes *dependent overlap*: activation all-gather feeds a tensor-parallel GEMM, and reduce-scatter consumes the GEMM output before returning to a sequence-sharded layout. Whole-operation execution would serialize these pairs, so systems expose overlap by chunking or tiling the dependent region and pipelining communication with computation [5, 12, 17, 20, 21, 23]. EP follows the same dependent-overlap idea at microbatch granularity: all-to-all dispatch/combine is on the producer-consumer path inside one MoE phase, so MoE systems overlap all-to-all from one microbatch with attention or expert FFN computation from a neighboring microbatch [7, 14, 26].

Across these parallelism schemes, collective communication lies on the critical path of distributed LLM execution. As model size, sequence length, and GPU count increase, the latency of all-gather, reduce-scatter, and all-to-all can limit scalability [15–17]. Modern systems therefore commonly use CC overlap, scheduling communication to run concurrently with useful computation so that communication latency is hidden from the critical path.

3. Characterizing Overlap Overhead

This section characterizes why exposed CC overlap does not directly translate into efficient concurrent execution. We first quantify overlap-induced overhead in LLaMA and Mixtral workloads implemented with Megatron-LM, asking whether the inefficiency observed in prior work appears across representative LLM parallelism settings. We then use targeted microbenchmarks derived from the overlap

patterns in Section 2.4 to isolate the hardware, system, and kernel mechanisms behind the overhead. This two-level structure connects end-to-end slowdown to hardware, system, and kernel behavior, avoiding a single-layer explanation of overlap overhead.

3.1. Experimental Setup and Metrics

End-to-end experiments. The end-to-end experiments use Megatron-LM with Transformer Engine. The main end-to-end workloads cover LLaMA and Mixtral models across training, inference prefill, dense, MoE, single-node, and multi-node settings. Table 1 summarizes the workload, scale, parallelism configuration, GPU platform, sequence length, end-to-end timing, and kernel-level slowdown. In both single-node and multi-node experiments, intra-node GPUs are connected by NVLink. In the multi-node experiments, NCCL inter-node communication runs over InfiniBand. These experiments measure overlap-induced slowdown, the sequential, overlap, and idealized end-to-end timings, and rank-wise straggler imbalance.

Targeted microbenchmarks. The targeted microbenchmarks run on the same single-node H100/H200 systems used for the single-node end-to-end experiments, with intra-node GPUs connected by NVLink. They instantiate the representative overlap patterns summarized in Fig. 3. FSDP contributes layer-level independent all-gather and reduce-scatter pairs with compute. TP/SP contributes chunk-level activation all-gather and reduce-scatter pairs with GEMM, and EP contributes microbatch-level all-to-all pairs with compute. For each pair, the compute side uses representative GEMM and non-GEMM kernels, implemented with cuBLAS and Transformer Engine C APIs for RMSNorm and SwiGLU, while the communication side uses NCCL all-gather, reduce-scatter, and all-to-all collectives. Table 2 lists the concrete representative kernel dimensions and collective message sizes, derived from LLaMA 3.1 8B for dense FSDP/TP/SP and from Mixtral 8×7B for EP.

Starting from these LLM-derived instances, we perform two analyses. First, we measure the resource usage of the representative compute and communication kernels, including launch configuration, register usage, and shared-memory usage, and report the combined demand of each compute–communication pair in Table 2. Second, for selected representative GEMM-overlap cases, we run sweep experiments to measure kernel-level slowdown as compute size, communication granularity, launch order, and stream priority vary. Table 3 summarizes the sweep configuration. Launch order refers to the host-side order of CUDA launch calls. In the **GEMM-first** order, the benchmark calls the GEMM launch before the collective launch. In the **communication-first** order, it calls the collective launch before the GEMM launch. Although non-GEMM experiments show the same qualitative slowdown trend, the detailed microbenchmark analysis focuses on GEMM-based overlap, where resource contention is strongest. Microbenchmark component times and slowdown ratios are measured with CUDA events, and the experiments

additionally collect Nsight Systems traces for queue-delay analysis. The communication-kernel decomposition experiment profiles Ring/Simple NCCL collectives with 32 thread blocks and decomposes primitive cycles into peer-wait time before data becomes available, reduce/copy work on the data path, and post-peer synchronization after issuing the transfer.

Metrics. For each end-to-end workload, we compare the iteration time under three cases, *sequential* (T_{seq}), *overlap* (T_{overlap}), and *idealized* (T_{ideal}). The sequential run keeps the framework overlap options enabled but sets `CUDA_LAUNCH_BLOCKING=1`, forcing CUDA launches and progress to serialize. The overlap run uses the same framework options with nonblocking CUDA launches, so the two traces differ only in whether the issued work is allowed to overlap at runtime. We compute the idealized iteration time (T_{ideal}) by replaying the overlapped trace as a dependency graph that preserves same-stream order and explicit CUDA event dependencies. In the replay, each matched kernel uses its no-overlap duration from the sequential run. Thus, T_{ideal} captures the overlap time that would remain if kernel-level slowdown were removed under the trace-visible scheduling dependencies. We do not infer hidden cross-stream dependencies that are not explicit in the trace.

For end-to-end kernel-level analysis, we compare each kernel type between the sequential and overlap runs using Nsight Systems kernel activity durations. The per-kernel slowdown is

$$s_k = \frac{t_{k,\text{overlap}} - t_{k,\text{seq}}}{t_{k,\text{seq}}}.$$

The aggregate kernel-level slowdown is computed as the occurrence-weighted average of s_k over the matched kernel types observed in the overlap trace. Here, kernel duration refers to the GPU execution interval after the kernel obtains residency, so it excludes pre-residency queueing delay.

Queue delay is the time after a communication kernel is launched but before it obtains SM residency. We estimate it as the difference between the CUDA-event elapsed time around the communication kernel and the Nsight Systems kernel execution span after thread blocks obtain SM residency:

$$t_{\text{queue}} = t_{\text{CUDA-event}} - t_{\text{NSYS}}.$$

For microbenchmark sweep results, GEMM and communication slowdown compare each overlapped CUDA-event duration with its independent CUDA-event duration. These component event durations include any pre-residency waiting that occurs in the corresponding stream. Total slowdown compares the observed overlapped pair duration with the ideal duration under perfect overlap:

$$s_{\text{total}} = \frac{t_{\text{pair,overlap}}}{\max(t_{\text{GEMM,indep}}, t_{\text{comm,indep}})}.$$

For straggler analysis, the duration gap is the difference between the slowest and fastest ranks for the same collective

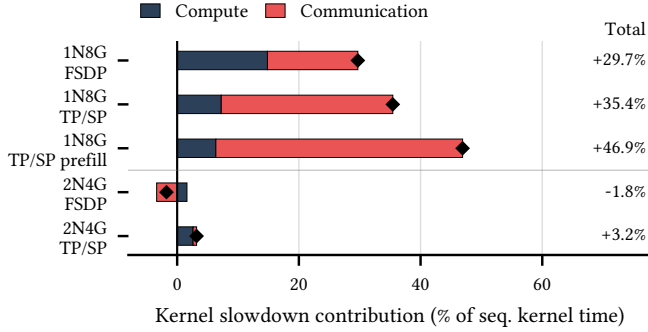


Figure 4: Compute and communication kernel slowdown decomposition for the LLaMA-3.1-8B workloads shown in Table 1. Contributions are signed and normalized by sequential matched-kernel time, excluding pre-residency queueing delay.

instance. For communication-kernel decomposition, each phase fraction is its cycle share within the profiled primitive.

3.2. End-to-End Overlap Overhead

Finding 1: CC overlap improves over sequential execution but remains far from ideal. Table 1 compares sequential, overlap, and ideal iteration time for the evaluated LLM workloads, with each workload normalized to its ideal time. Across these workloads, overlap reduces iteration time by 20.7–70.5% relative to sequential execution, confirming that communication hiding is effective. However, the overlap time still runs 1.04–4.46 $\times$ slower than the ideal. This gap is the runtime cost of realizing overlap, including kernel slowdown and residual scheduling bubbles.

Finding 2: single-node overlap efficiency is limited by kernel slowdown. Fig. 4 reports the corresponding compute and communication kernel slowdown decomposition. This view separates the end-to-end benefit of overlap from the cost paid by kernels after they start executing on the GPU. In the single-node LLaMA and Mixtral workloads, matched-kernel time increases by 20.4–55.8%, with communication kernels contributing most of the slowdown in TP/SP, inference prefill, and Mixtral EP8. This confirms that the end-to-end gap to the ideal is not only a scheduling artifact: in single-node overlap, the kernels participating in the overlap window themselves slow down after they begin executing.

Finding 3: multi-node overlap efficiency is limited by compute–communication duration imbalance. The

two-node H100 runs show a different result from the single-node cases. Kernel slowdown is much smaller. FSDP is near zero overall, while the TP4+SP+DP2 run increases matched-kernel time by only 3.2%. However, the multi-node overlap time still remains far from the ideal, showing that small kernel slowdown does not imply efficient overlap.

The gap comes from compute–communication duration imbalance. Inspecting the profiled LLaMA traces shows that NCCL uses a grid size of only 4 for inter-node collectives, compared with grid sizes of 24–32 for the single-node collectives in the representative overlap pairs. This appears to come from NCCL’s topology-aware grid selection. For inter-node communication, NCCL adjusts the communication grid to the bandwidth that the interconnect can sustain. The smaller grid reduces SM occupancy and memory-system pressure, so resource-contention-induced kernel slowdown becomes small. However, it also lengthens the communication kernel. As a result, communication is not effectively hidden by overlap and remains exposed on the critical path, limiting the performance benefit of overlap.

The two multi-node configurations support this interpretation. In the two-node TP4+SP+DP2 run, TP and SP collectives remain intra-node and only the data-parallel dimension crosses nodes, so its behavior is closer to the single-node cases than two-node FSDP, where the FSDP data-parallel group spans all eight GPUs and exposes more inter-node collective communication. This contrast shows that efficient overlap must account for compute–communication duration imbalance, not merely the amount of concurrency created.

This also suggests a possible optimization direction. When communication is exposed on the critical path, starting communication earlier by decomposing compute into finer scheduling units may be beneficial, even if doing so introduces additional compute slowdown.

Together, the end-to-end results show that missed overlap benefit does not come from a single source. In single-node workloads, the gap to the ideal is closely tied to kernel slowdown after compute and communication execute concurrently. In multi-node workloads, overlap can remain inefficient even when kernel slowdown is small if communication outlasts compute, creating an execution-time imbalance that exposes residual communication on the critical path. The following analyses examine these effects across layers: hardware resource contention, runtime scheduling and collective imbalance, and communication-kernel execution behavior.

3.3. Hardware Perspective: Resource Contention

This subsection analyzes the resource contention that appears in hardware when CC overlap is enabled. The analysis focuses on SM residency, which determines whether compute and communication thread blocks can execute simultaneously, and memory-system contention, which determines how they interfere after obtaining residency.

Finding 4: CC overlap is re-serialized in hardware and creates communication starvation. Multi-stream

TABLE 1: END-TO-END AND KERNEL-LEVEL OVERLAP SUMMARY FOR EVALUATED LLM WORKLOADS.

Workload	Scale	Parallelism	GPU	Seq.	$T_{\text{seq}}/T_{\text{ideal}}$	$T_{\text{overlap}}/T_{\text{ideal}}$	Kernel slowdown
LLaMA-3.1-8B train	1N8G	FSDP	H100	8192	1.80 $\times$	1.31 $\times$	+29.7%
LLaMA-3.1-8B train	1N8G	TP8+SP	H100	8192	8.56 $\times$	4.46 $\times$	+35.4%
LLaMA-3.1-8B train	1N8G	TP4+SP+DP2	H100	8192	4.96 $\times$	2.99 $\times$	+20.4%
LLaMA-3.1-8B prefill	1N8G	TP8+SP	H100	8192	3.51 $\times$	1.04 $\times$	+46.9%
Mixtral-8 $\times$ 7B train	1N8G	FSDP+EP8	H200	1024	5.79 $\times$	2.74 $\times$	+55.8%
LLaMA-3.1-8B train	2N4G	FSDP	H100	8192	2.71 $\times$	2.00 $\times$	-1.8%
LLaMA-3.1-8B train	2N4G	TP4+SP+DP2	H100	8192	1.87 $\times$	1.48 $\times$	+3.2%

TABLE 2: REPRESENTATIVE CC OVERLAP INSTANCES DERIVED FROM LLAMA 3.1 8B AND MIXTRAL 8×7B.

Pair	Src.	Compute kernel	Compute dim.	Compute grid/block	Comm.	Msg. size	Comm. grid/block	Reg. (comp.+comm.)	SMEM (comp.+comm.)
AG GEMM	FSDP	QKV proj. fwd	8192×6144×4096	2×66, 384	Param. AG	54.5 MB	24, 544	98.4+79.7=178.1%	81.2+44.5=125.7%
AG GEMM	TP/SP	QKV proj. chunk	1024×768×4096	2×64, 384	Act. AG	8.4 MB	24, 544	98.4+79.7=178.1%	91.8+44.5=136.3%
AG RMSNorm	FSDP	RMSNorm bwd	8192×4096	128, 1024	Param. AG	54.5 MB	24, 544	40.6+79.7=120.3%	4.1+44.5=48.6%
RS GEMM	FSDP	FFN down dgrad GEMM	8192×14336×4096	2×66, 384	Grad. RS	109.1 MB	24, 544	98.4+79.7=178.1%	81.2+44.5=125.7%
RS GEMM	TP/SP	FFN down proj. chunk	2048×4096×1792	2×66, 384	Act. RS	2.1 MB	24, 640	98.4+93.8=192.2%	91.8+44.5=136.2%
RS SwiGLU	FSDP	SwiGLU bwd	8192×14336	14336, 512	Grad. RS	109.1 MB	24, 544	35.9+79.7=115.6%	0.5+44.5=45.0%
A2A GEMM	EP	FFN gate/up proj. fwd	8192×28672×4096	2×66, 384	Act. A2A	67.1 MB	32, 640	98.4+93.8=192.2%	81.2+44.5=125.7%
A2A RMSNorm	EP	RMSNorm fwd	4096×4096	924, 128	Act. A2A	67.1 MB	32, 640	13.9+93.8=107.7%	0.5+44.5=45.0%
A2A SwiGLU	EP	SwiGLU fwd	8192×14336	14336, 512	Act. A2A	67.1 MB	32, 640	30.5+93.8=124.3%	0.5+44.5=45.0%

AG = all-gather, RS = reduce-scatter, A2A = all-to-all.

TABLE 3: TARGETED MICROBENCHMARK SWEEP.

Side	Instance	Sweep
Compute	GEMM	$(M, N, K) = (T, 28672, 4096)$
Compute	RMSNorm	$[T, 4096]$
Compute	SwiGLU	$[T, 28672] \rightarrow [T, 14336]$
Comm.	AG, RS, A2A	thread blocks = 4–32, 16 MB/block
Comm.	AG, RS, A2A	thread blocks = 32, msg. = 64–512 MB

AG = all-gather, RS = reduce-scatter, A2A = all-to-all.

TABLE 4: H100/H200 GPU RESOURCES AND PER-SM RESIDENCY LIMITS.

Category	Item	H100	H200
GPU resource	SMs	132	132
	L2 cache	50 MB	50 MB
	DRAM capacity	80 GB	141 GB
Per-SM limit	Threads	2048	2048
	Thread blocks	32	32
	Registers	65,536 32-bit	65,536 32-bit
	Shared memory	228 KiB	228 KiB

execution allows the host to issue compute and communication kernels concurrently. However, a launched kernel can execute on the GPU only after its thread blocks are assigned to SMs. Therefore, multi-stream execution does not by itself guarantee concurrent execution in hardware.

For two kernels to execute concurrently at the hardware level, their thread blocks must be scheduled in one of two ways. First, their thread blocks can occupy different SMs. However, if the sum of the two grid sizes exceeds the number of SMs, some thread blocks cannot be assigned immediately and must wait until SMs become available. Second, thread blocks from both kernels can share the same SM. This requires the combined per-thread-block demand for threads, registers, shared memory, and resident block slots to fit within the per-SM limits. Table 4 summarizes the H100/H200 SM count and per-SM limits used for this check.

Table 2 reports the representative CC overlap pairs used in the characterization. The compute grid/block and communication grid/block columns report the launch geometry of each kernel, where grid size is the number of thread blocks and block size is the number of threads per block. Reg. and SMEM list compute+communication demand normalized to the per-SM limits in Table 4. Totals above 100% rule out same-SM co-residency. The table shows that representative LLM overlap pairs often cannot co-reside on the same SM. GEMM thread blocks consume nearly the full register budget and most of the shared-memory budget. Non-GEMM kernels use less shared memory, but still consume enough registers that they cannot be paired with NCCL on the same SM. NCCL thread blocks use about half of the shared-memory budget and 52.2–61.4K registers, so combining them with the compute kernels exceeds at least one per-SM limit in most cases.

Consequently, kernels used in representative LLM overlap patterns cannot share an SM. Compute kernels are typically tuned for standalone throughput on the target GPU architecture, using large grids and high per-block resource demand to fill the device. Therefore, simply moving to another GPU architecture does not remove the issue. Furthermore, if the compute kernel obtains SM residency first, communication blocks from another stream may fail to receive SM allocation and experience starvation until compute blocks release SM slots. This same-SM co-residency failure can turn exposed overlap into hardware-level re-serialization. Fig. 5 shows this directly. With GEMM launched first, communication queue delay reaches 2.07ms for all-gather, 2.07ms for reduce-scatter, and 2.06ms for all-to-all at $M = 8192$. Normalized by the communication-kernel execution span, this corresponds to 37.1% for all-gather, 38.5% for reduce-scatter, and 211% for all-to-all. Thus, system designs that assume overlap from host-side concurrent launch must also check whether the issued kernels can actually be overlapped on the GPU.

Finding 5: SM contention slows kernels after residency is obtained. Re-serialization is not the only problem at the SM level. When communication is issued first, or when resident compute blocks finish and free SM slots, NCCL communication blocks can obtain SM residency

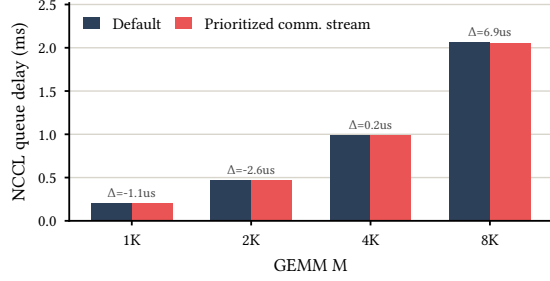


Figure 5: Communication-kernel queue delay, $t_{\text{CUDA-event}} - t_{\text{SYS}}$, in GEMM-first overlap experiments.

and execute concurrently with compute blocks. This is real hardware overlap, where compute and communication occupy different SMs at the same time. However, because the number of SMs is finite, each kernel receives fewer SMs than it would during independent execution. This shared SM allocation slows the participating kernels and makes the realized overlap fall short of the ideal latency-hiding bound.

The launch-order sweep in Fig. 8 shows this effect after the worst starvation case is avoided. In communication-first execution, the communication kernel obtains residency first and its slowdown remains near baseline, but the GEMM runs with fewer available SMs and slows down by up to $1.72\times$ for all-gather, $1.59\times$ for reduce-scatter, and $1.88\times$ for all-to-all. Therefore, even when overlap succeeds in creating concurrent GPU execution, SM sharing itself becomes a source of kernel slowdown.

Finding 6: memory contention remains after kernels obtain SMs. Even if software or scheduling changes mitigate SM contention, overlap still faces memory-system contention. Since compute and communication kernels share DRAM bandwidth and L2 cache, their combined memory demand can become a limiting factor after both kernels obtain SM residency. The sweep summarized in Table 3 shows this pressure in two representative cases. For gate/up GEMM, standalone L2 demand is already 67.2–77.1% of peak, and adding the largest all-gather and reduce-scatter configurations raises the summed L2 demand to 88.7–99.6% of peak. For memory-intensive SwiGLU, standalone DRAM demand reaches 75.9–90.5% of peak, and adding the largest all-gather and reduce-scatter configurations raises the summed DRAM demand above peak. These results show that after kernels obtain SM residency, DRAM and L2 bandwidth can remain limiting resources.

Fig. 6 examines this effect under communication-first overlap, where the communication kernel obtains SM residency before the GEMM and the worst communication starvation case is removed. With smaller GEMMs, communication slowdown for all-gather and reduce-scatter stays close to $1.00\times$, indicating that SM starvation dominates those cases. As GEMM size grows, however, memory pressure increases and communication slowdown becomes visible even without starvation. At the largest GEMM size,

average slowdown reaches $1.03\times$ for all-gather, $1.06\times$ for reduce-scatter, and $1.17\times$ for all-to-all, with individual configurations rising up to $1.32\times$. This slowdown shows that obtaining SM residency does not remove all interference. This also constrains the design space because techniques such as kernel fusion or finer software scheduling may reduce SM contention, but they can expose DRAM and L2 as the next bottleneck unless the hardware memory system is considered together with the software mechanism.

3.4. System Perspective: Runtime Effects

This subsection moves from per-GPU resource contention to system-level effects. We examine how rank-wise straggler effects, host launch order, and stream priority are related to CC overlap slowdown.

Finding 7: CC overlap amplifies straggler effects. A collective communication operation is constrained by the slowest rank, so rank-wise timing differences appear as straggler effects. As shown in Section 3.3, CC overlap increases the complexity of GPU execution by changing SM allocation and introducing resource contention after kernels obtain residency. These effects can differ across ranks. Consequently, ranks can make different amounts of communication progress while compute and communication are overlapped, widening the duration gap between the fastest and slowest ranks.

Fig. 7 shows representative single-node FSDP LLaMA training traces. In these traces, enabling overlap consistently increases both the average and P95 duration gap across all three models. The average gap grows by up to $5.38\times$ on LLaMA 3.1 8B, and the P95 gap reaches 5.248ms. Thus, overlap does not merely shift collective timing. It amplifies straggler effects across ranks and contributes to CC overlap slowdown.

Finding 8: host launch order determines realized overlap efficiency. A simple system-level way to reduce the SM starvation in Finding 4 is to change the host launch order. Launch order has a significant effect on CC overlap time because the first-issued kernel is more likely to obtain SM residency first. When compute is launched

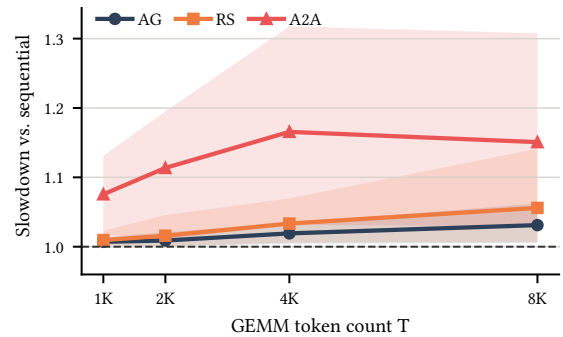


Figure 6: Communication slowdown in communication-first launches. Values are normalized to sequential communication execution. Shading shows the min-max range.

first, it can fill the GPU and force the communication kernel to wait, creating re-serialization. When communication is launched first, contention remains, but communication can obtain residency before compute fills the GPU, enabling real concurrent execution.

Fig. 8 quantifies the difference. Under GEMM-first execution, communication starvation propagates to pair-level overlap: total slowdown averages $1.39\times$ and reaches $1.96\times$ relative to ideal case. Under communication-first execution, the pair-level penalty is much smaller, with total slowdown averaging $1.07\times$ and peaking at $1.37\times$. Thus, overlap efficiency depends not only on whether compute and communication are issued together, but also on which kernel obtains SM residency first.

Finding 9: stream priority cannot recover missed residency. CUDA provides stream priority as a mechanism for favoring important work, so a natural response is to raise the communication stream priority. However, stream priority affects which thread block is selected at future scheduling points. It does not preempt thread blocks that are already resident on SMs. Therefore, if a large compute kernel has already filled the GPU, a higher-priority communication stream still has to wait for SM slots to become available.

Fig. 5 confirms this limitation. In GEMM-first experiments, prioritizing the communication stream changes the measured communication queue delay by only a few microseconds across GEMM sizes. The dominant factor is still whether the communication kernel was launched early enough to obtain residency before the compute kernel filled the GPU. Stream priority is useful only at scheduling boundaries and cannot by itself undo the re-serialization created by host launch order and resident compute blocks.

3.5. Kernel Perspective: Internal Execution Inefficiency

This subsection examines communication-kernel behavior after SM residency. We decompose NCCL primitive cycles to distinguish payload movement from synchronization and waiting inside the communication kernel.

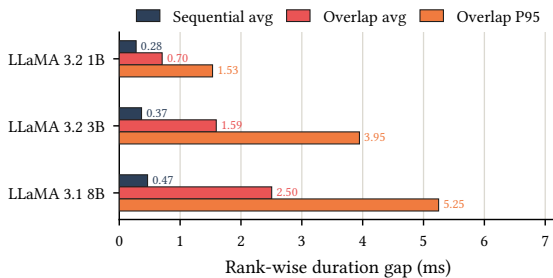


Figure 7: Rank-wise collective duration gap in single-node FSDP LLaMA runs. Bars report sequential average, overlap average, and overlap P95 gaps.

Finding 10: overlap increases communication-kernel inefficiency. An NCCL collective transfers data through a repeated sequence of peer-wait, reduce/copy, and post-peer phases [31]. The reduce/copy phase moves payload data and performs the required copy or reduction. In contrast, peer-wait and post-peer are synchronization phases. During peer-wait, the communication kernel polls synchronization variables until the next slice becomes available. The thread block remains resident on an SM during this polling, but it is not moving payload data. In other words, the communication kernel continues to occupy SM resources while the SM’s execution capability is largely unused.

Overlap increases this non-payload resident time. Fig. 9 shows the decomposition using the $M = 8192$ gate/up GEMM for GEMM-first overlap. Under overlap, the communication kernel spends a larger fraction of its SM-resident time in peer-wait and a smaller fraction in reduce/copy. For example, all-gather increases its peer-wait fraction from 7.4% in isolation to 27.4% under GEMM-first overlap, and the same shift toward waiting appears across the other collectives.

This inefficiency is especially costly under CC overlap. As Finding 5 shows, communication blocks can slow compute by occupying SMs that compute would otherwise use. When those communication blocks spend a large fraction of SM-resident time polling for peer progress, they consume SM execution capacity while advancing data movement less efficiently. The effect can also grow with the rank-wise timing variation discussed in Finding 7. If overlap makes ranks progress at different rates, communication kernels spend more time waiting for peer-visible progress. These results suggest that improving CC overlap requires reducing SM-resident polling time, for example through signaling or other progress mechanisms, or by restructuring compute and communication with fine-grained fused kernels [20, 21, 29, 30].

Together, these findings show that maximizing overlap opportunity alone does not address the inefficiencies hidden in cross-layer execution. The observed overhead emerges from the interaction between hardware resource allocation, runtime scheduling, collective timing, and kernel behavior. Thus, efficient CC overlap requires hardware-software co-design that accounts for the efficiency of concurrent execution in order to translate overlap opportunities into actual performance gains.

4. Discussion

The characterization results suggest that improving CC overlap requires optimizing how overlap is realized, not only exposing more overlap opportunities. We distill the results into two design probes, kernel resource usage control for SM sharing and asynchronous communication data movement. We do not present these probes as complete optimizations. Instead, each follows from a characterization finding, addresses one part of the problem, and exposes why a single-layer fix is insufficient.

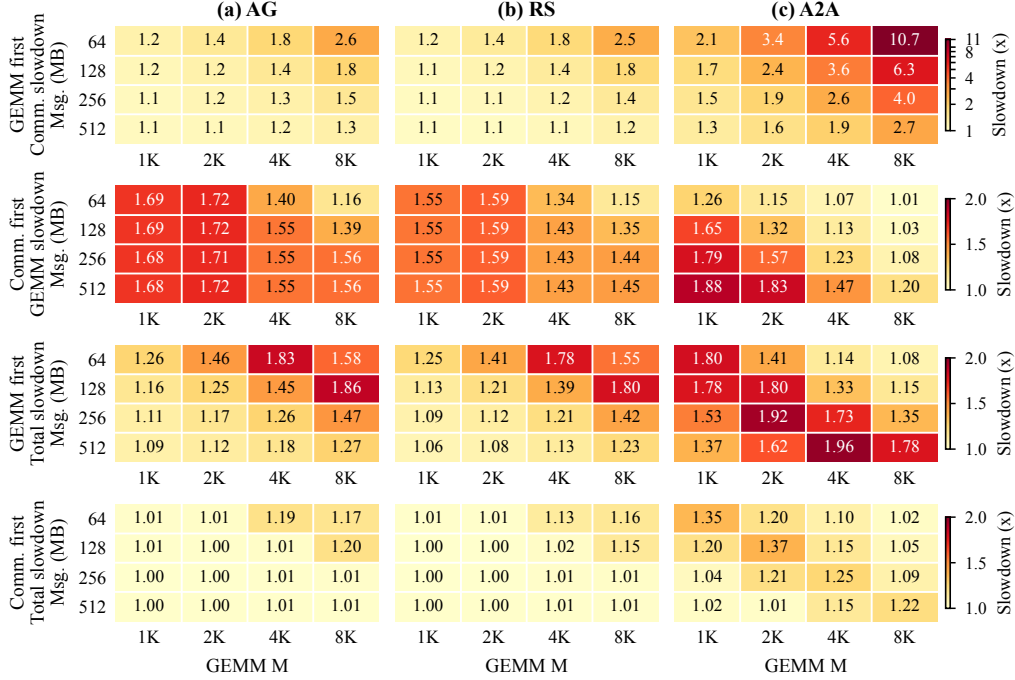


Figure 8: Execution-order sensitivity in GEMM-collective overlap. The top two rows report component-level slowdown, and the bottom two rows report total slowdown relative to perfect overlap.

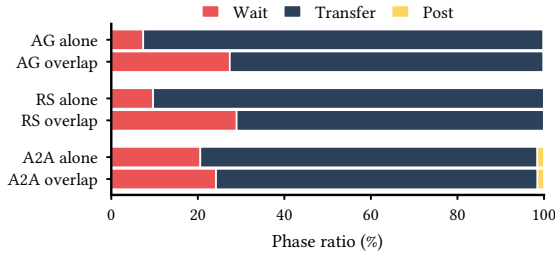


Figure 9: NCCL primitive cycle breakdown for 256 MB collectives with a communication grid size of 32.

Kernel resource usage control for SM sharing. The SM residency results show that starvation and re-serialization originate from high kernel resource demand. Compute and communication blocks often cannot occupy the same SM, so they must be placed on different SMs, and a large compute grid can delay communication until SM slots become available. A natural response is to reshape kernel resource usage so compute and communication blocks can share the same SM. Our probe follows this direction by reducing the per-block footprint of the compute kernel. It reduces queueing delay and enables same-SM sharing, but the benefit does not translate into better overall overlap efficiency. Sharing one SM reduces the resources available to each kernel. For compute, this reduces tiling efficiency and Tensor Core utilization, causing larger per-kernel slowdown. This suggests that SM sharing cannot be treated as a

software-only afterthought. Hardware resource provisioning and kernel resource allocation must be designed with multi-kernel SM residency in mind.

Asynchronous communication data movement. The communication-kernel and memory-contention results point to a second direction. Under overlap, communication blocks can occupy SMs while polling, and compute and communication also contend for DRAM and L2 bandwidth. We therefore explored whether communication could pipeline GPU-internal data movement with inter-rank transfer. Hopper’s Tensor Memory Accelerator (TMA) and L2 prefetching provide two probes for this direction. TMA can move data asynchronously into staging buffers, while L2 prefetching tries to prepare communication data before the collective consumes it. These mechanisms improve the data-movement portion in isolation. With TMA, the measured transfer portion is reduced by up to $4.0\times$ at the same transfer granularity.

However, the end-to-end overlap benefit remains limited. TMA staging adds synchronization and pipeline-management overhead, and its shared-memory staging granularity limits how much data can be moved per step. Deeper pipelining would require additional shared memory for staging, but the current shared-memory capacity cannot provide enough in-flight staging to sustain high throughput without tightening the occupancy tradeoff. These synchronization points again create underutilized SM-resident time, similar in effect to NCCL polling, and are sensitive to rank-to-rank progress variation. L2 prefetching has a related limitation. Receive/FIFO data can be

prefetched only after `waitPeer` validates the current slice, communication payloads are mostly streaming with little reuse, and prefetch requests consume the same L2 capacity and memory bandwidth shared with compute. Multi-node communication adds another constraint because the inter-node link can become the dominant bottleneck. In that regime, filling local memory bandwidth more efficiently is not enough. These results suggest that communication progress, memory-system support, architectural resource provisioning, and network-level compute-communication balance must be designed together.

Overall, no single layer can make CC overlap efficient. Runtime scheduling can reduce re-serialization, kernel and communication mechanisms can reduce SM-resident waiting, and hardware support can reduce memory-system contention, but these choices interact. Future designs should therefore optimize realized overlap efficiency across the hardware, runtime, and kernel layers.

5. Related Work

5.1. Maximizing CC Overlap Opportunity

Most recent CC overlap work focuses on creating more opportunities for communication and computation to run concurrently. Runtime systems such as PyTorch DDP and FSDP use separate CUDA streams and events to launch communication asynchronously while preserving data dependencies [18, 19]. Other systems decompose communication, computation, or execution graphs into smaller units, then reorder or pipeline them to expose more concurrent execution windows [17, 22–25, 35, 36].

A related line of work moves the overlap boundary inside kernels. Kernel fusion and fine-grained overlap mechanisms combine communication with GEMM, MoE, embedding, or pipeline-stage work, or trigger communication from GPU-side progress points [20, 21, 26, 30, 37–39]. These mechanisms reduce the limitations of coarse multi-stream scheduling, but they primarily increase overlap opportunity. Our work is complementary: after overlap has been exposed, we characterize how efficiently the resulting concurrent execution is realized on the GPU.

5.2. Characterizing CC Overlap Overhead

Prior work has also shown that CC overlap can introduce overhead. Some studies quantify the overhead visible in end-to-end distributed deep learning [28], while others identify resource contention between collective communication and computation or explore prioritization, partitioning, and offload mechanisms [27, 29]. Additional work mitigates specific interference sources through communication-thread allocation, kernel-interference profiling, or low-level NCCL tuning [40–42].

These studies establish that overlap is not free, but they typically focus on a particular scope, such as visible end-to-end overhead, a specific resource-control mechanism, or low-level communication tuning. In contrast, our goal is not to introduce another overlap mechanism or tune one

communication primitive. We explain why exposed overlap is not efficiently realized in modern GPU LLM workloads by connecting end-to-end slowdown with targeted microbenchmarks. This lets us separate architectural constraints, such as SM residency limits and shared DRAM/L2 resources, from implementation-dependent communication behavior, such as NCCL grid-size selection, launch order, stream priority, and polling-based progress.

6. Conclusion

CC overlap is a common technique for hiding communication in distributed LLM workloads, but this paper shows that concurrent launch does not guarantee efficient concurrent execution. Our end-to-end LLaMA and Mixtral experiments show that overlap inefficiency appears in different forms across system settings. In the evaluated single-node LLM workloads, overlap introduces 20.4–55.8% kernel slowdown relative to sequential execution. In multi-node workloads, kernel slowdown can be small, yet overlap can still remain up to $2.00\times$ slower than ideal because communication outlasts compute and remains exposed on the critical path. These results show that distributed LLM systems must consider overlap efficiency, not only the number of overlap opportunities they create.

Our microbenchmarks explain why this inefficiency persists across layers. At the hardware level, SM contention can re-serialize concurrently issued kernels and slow them down, and memory bandwidth contention remains even after kernels obtain SM residency. At the system level, launch order, stream priority limits, and rank-wise straggler effects determine how much of this contention becomes visible in execution time. At the kernel level, communication kernels can spend substantial SM-resident time in polling and peer-wait phases, consuming compute resources without advancing payload movement. These results suggest that single-layer optimization alone is insufficient, because a bottleneck mitigated at one layer can reappear at another layer.

Future systems should therefore optimize CC overlap efficiency, not only overlap opportunity, across the hardware, runtime, and kernel layers. This requires runtimes, kernels, and GPU architectures to be designed together with compute-communication co-execution in mind.

Acknowledgment

The authors used generative AI tools only for grammar editing and template compliance checking. This research was supported through cyber-infrastructure research resources and services provided by the Partnership for an Advanced Computing Environment (PACE) at the Georgia Institute of Technology. This work is supported in part by NSF grant number IIS-2544321 and gifts from Google, AMD, and Samsung.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [3] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Židek, A. Potapenko *et al.*, "Highly accurate protein structure prediction with alphafold," *nature*, vol. 596, no. 7873, pp. 583–589, 2021.
- [4] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020. [Online]. Available: <https://arxiv.org/abs/2001.08361>
- [5] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020. [Online]. Available: <https://arxiv.org/abs/1909.08053>
- [6] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "Palm: Scaling language modeling with pathways," 2022. [Online]. Available: <https://arxiv.org/abs/2204.02311>
- [7] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [8] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzie, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O'Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorný, M. Pokrass, V. H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P. Tillet, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, "Gpt-4 technical report," 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [9] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Young, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpanogiannis, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou,

- R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baeviski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Sweet, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damla, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [10] J. Dean, G. S. Corrado, R. Monga, K. Chen, M. Devin, Q. V. Le, M. Z. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, and A. Y. Ng, "Large scale distributed deep networks," in *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS'12. Red Hook, NY, USA: Curran Associates Inc., 2012, pp. 1223–1231.
- [11] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," 2020. [Online]. Available: <https://arxiv.org/abs/1910.02054>
- [12] V. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoenybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," 2022. [Online]. Available: <https://arxiv.org/abs/2205.05198>
- [13] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," 2019. [Online]. Available: <https://arxiv.org/abs/1811.06965>
- [14] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," 2020. [Online]. Available: <https://arxiv.org/abs/2006.16668>
- [15] Z. Zhang, C. Chang, H. Lin, Y. Wang, R. Arora, and X. Jin, "Is network the bottleneck of distributed training?" 2020. [Online]. Available: <https://arxiv.org/abs/2006.10103>
- [16] S. Shi, Z. Tang, X. Chu, C. Liu, W. Wang, and B. Li, "A quantitative survey of communication optimizations in distributed deep learning," *IEEE Network*, vol. 35, no. 3, pp. 230–237, 2020.
- [17] G. Wang, C. Zhang, Z. Shen, A. Li, and O. Ruwase, "Domino: Eliminating communication in llm training via generic tensor slicing and overlapping," 2024. [Online]. Available: <https://arxiv.org/abs/2409.15241>
- [18] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li, "Pytorch fsdp: Experiences on scaling fully sharded data parallel," 2023. [Online]. Available: <https://arxiv.org/abs/2304.11277>
- [19] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: Experiences on accelerating data parallel training," 2020. [Online]. Available: <https://arxiv.org/abs/2006.15704>
- [20] L.-W. Chang, W. Bao, Q. Hou, C. Jiang, N. Zheng, Y. Zhong, X. Zhang, Z. Song, C. Yao, Z. Jiang, H. Lin, X. Jin, and X. Liu, "Flux: Fast software-based communication overlap on gpus through kernel fusion," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06858>
- [21] S. Zheng, J. Fang, X. Zheng, Q. Hou, W. Bao, N. Zheng, Z. Jiang, D. Wang, J. Ye, H. Lin, L.-W. Chang, and X. Liu, "Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives," 2025. [Online]. Available: <https://arxiv.org/abs/2503.20313>
- [22] S. Cheng, S. Lin, L. Diao, H. Wu, S. Wang, C. Si, Z. Liu, X. Zhao, J. Du, W. Lin, and Y. You, "Concerto: Automatic communication optimization and scheduling for large-scale deep learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 198–213. [Online]. Available: <https://doi.org/10.1145/3669940.3707223>
- [23] C. Chen, X. Li, Q. Zhu, J. Duan, P. Sun, X. Zhang, and C. Yang, "Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 178–191. [Online]. Available: <https://doi.org/10.1145/3620666.3651379>
- [24] R. Gond, N. Kwatra, and R. Ramjee, "Tokenweave: Efficient compute-communication overlap for distributed llm inference," 2026. [Online]. Available: <https://arxiv.org/abs/2505.11329>
- [25] B. Xiao and L. Su, "Iso: Overlap of computation and communication within sequence for llm inference," 2024. [Online]. Available: <https://arxiv.org/abs/2409.11155>

- [26] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu, "Comet: Fine-grained computation-communication overlapping for mixture-of-experts," 2025. [Online]. Available: <https://arxiv.org/abs/2502.19811>
- [27] S. Rashidi, M. Denton, S. Sridharan, S. Srinivasan, A. Suresh, J. Nie, and T. Krishna, "Enabling compute-communication overlap in distributed deep learning training platforms," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Jun. 2021, pp. 540–553. [Online]. Available: <http://dx.doi.org/10.1109/ISCA52012.2021.00049>
- [28] S. Lee, J. Oh, J. Kim, S. Go, J. Park, and D. Mahajan, "Characterizing compute-communication overlap in gpu-accelerated distributed deep learning: Performance and power implications," 2025. [Online]. Available: <https://arxiv.org/abs/2507.03114>
- [29] A. Agrawal, S. Aga, S. Pati, and M. Islam, "Optimizing ml concurrent computation and communication with gpu dma engines," 2025. [Online]. Available: <https://arxiv.org/abs/2412.14335>
- [30] K. Hong, X. Li, M. Liu, Q. Mao, T. Wu, Z. Huang, L. Chen, Z. Wang, Y. Zhang, Z. Zhu, G. Dai, and Y. Wang, "Efficient and adaptable overlapping for computation and communication via signaling and reordering," in *Proceedings of the 21st European Conference on Computer Systems*, ser. EUROSYS '26. ACM, Apr. 2026, pp. 1894–1911. [Online]. Available: <http://dx.doi.org/10.1145/3767295.3769370>
- [31] NVIDIA, "NVIDIA Collective Communication Library (NCCL) Documentation," 2026, accessed: 2026-05-15. [Online]. Available: <https://docs.nvidia.com/deeplearning/nccl>
- [32] E. Warraich, O. Shabtai, K. Manaa, S. Vargaftik, Y. Piasetzky, M. Kadosh, L. Suresh, and M. Shahbaz, "Optireduce: Resilient and tail-optimal allreduce for distributed deep learning in the cloud," 2025. [Online]. Available: <https://arxiv.org/abs/2310.06993>
- [33] J. Lin, Z. Jiang, Z. Song, S. Zhao, M. Yu, Z. Wang, C. Wang, Z. Shi, X. Shi, W. Jia, Z. Liu, S. Wang, H. Lin, X. Liu, A. Panda, and J. Li, "Understanding stragglers in large model training using what-if analysis," 2025. [Online]. Available: <https://arxiv.org/abs/2505.05713>
- [34] A. Devraj, E. Ding, A. V. Kumar, R. Kleinberg, and R. Singh, "Efficient allreduce with stragglers," 2025. [Online]. Available: <https://arxiv.org/abs/2505.23523>
- [35] S. Wang, J. Wei, A. Sabne, A. Davis, B. Ilbeyi, B. Hechtman, D. Chen, K. S. Murthy, M. Maggioni, Q. Zhang, S. Kumar, T. Guo, Y. Xu, and Z. Zhou, "Overlap communication with dependent computation via decomposition in large deep learning models," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 93–106. [Online]. Available: <https://doi.org/10.1145/3567955.3567959>
- [36] Y. Gao, B. Hu, M. Boloursaz Mashhadi, W. Wang, R. Tafazolli, and M. Debbah, "Pipedap: An efficient communication framework for scheduling decoupled all-reduce primitives in distributed dnn training," *IEEE Transactions on Emerging Topics in Computing*, 2025.
- [37] K. Punniyamurthy, K. Hamidouche, and B. M. Beckmann, "Optimizing distributed ml communication with fused computation-collective operations," 2023. [Online]. Available: <https://arxiv.org/abs/2305.06942>
- [38] O. J. Aimuyo, B. Oh, and R. Singh, "Flashmoe: Fast distributed moe in a single kernel," 2025. [Online]. Available: <https://arxiv.org/abs/2506.04667>
- [39] Y. Zhong, Z. Zhang, B. Wu, S. Liu, Y. Chen, C. Wan, H. Hu, L. Xia, R. Ming, Y. Zhu, and X. Jin, "Optimizing RLHF training for large language models with stage fusion," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. Philadelphia, PA: USENIX Association, Apr. 2025, pp. 489–503. [Online]. Available: <https://www.usenix.org/conference/nsdi25/presentation/zhong>
- [40] Y. Liu, B. Jiang, S. Zhao, T. Lin, X. Wang, and C. Zhou, "Libra: Contention-aware gpu thread allocation for data parallel training in high speed networks," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, 2023.
- [41] K. Zhu, Y. Gao, Y. Zhao, L. Zhao, G. Zuo, Y. Gu, D. Xie, T. Tang, Q. Xu, Z. Ye, K. Kamahori, C.-Y. Lin, Z. Wang, S. Wang, A. Krishnamurthy, and B. Kasikci, "Nanoflow: Towards optimal large language model serving throughput," 2024. [Online]. Available: <https://arxiv.org/abs/2408.12757>
- [42] G. Xu, Z. Le, Y. Chen, Z. Lin, Z. Jin, Y. Miao, and C. Li, "AutoCCL: Automated collective communication tuning for accelerating distributed and parallel DNN training," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. Philadelphia, PA: USENIX Association, Apr. 2025, pp. 667–683. [Online]. Available: <https://www.usenix.org/conference/nsdi25/presentation/xu-guanbin>